



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Kurt Millares Tyrael Lance Peralta
Section:	AN42
Professor:	Crisola Tan
Date:	10/05/2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware

- A laptop or desktop computer
- Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)

- Google Drive for shared documentation
- GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
(Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.



Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.

Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

- ✓ Similarity score or ranking
- ✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.

This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

- ✓ "New case saved" or similar confirmation
- ✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:

After final verdict, the case is stored—enhancing the system’s knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.

Student submits a math solution with repeated errors.

System retrieves past students with similar error patterns (e.g., misusing formulas).

It adapts feedback and personalized lessons based on successful interventions.

Learning:

System improves by retaining new student profiles and outcomes.

A.

Your Code - Laboratory Tasks



```
# =====  
# INTELLIGENT TUTORING SYSTEM  
# Case-Based Reasoning (CBR)  
# Dynamic User Input Version  
# =====  
  
# -----  
# 1. CASE BASE: PAST STUDENT CASES  
# -----  
  
case_base = [  
    {  
        "case_id": 1,  
        "problem": {  
            "topic": "quadratic equations",  
            "error_pattern": ["wrong formula", "sign error"],  
            "difficulty": "medium",  
            "score": 55  
        },  
        "solution": {  
            "feedback": [  
                "Review the quadratic formula.",  
                "Check positive and negative signs carefully."  
            ],  
            "lesson": ["Step-by-step quadratic formula practice"],  
            "intervention": [  
                "Worked examples followed by guided practice."  
            ]  
        }  
    },  
    {  
        "case_id": 2,  
        "problem": {  
            "topic": "quadratic equations",  
            "error_pattern": ["wrong formula", "substitution error"],  
            "difficulty": "medium",  
            "score": 60  
        }  
    }  
]
```

```

    },
    "solution": {
      "feedback": [
        "Identify a, b, and c before using the formula."
      ],
      "lesson": [
        "Identifying coefficients in quadratic equations"
      ],
      "intervention": [
        "Formula identification exercises."
      ]
    }
  },
  {
    "case_id": 3,
    "problem": {
      "topic": "linear equations",
      "error_pattern": ["transposition error", "sign error"],
      "difficulty": "easy",
      "score": 65
    },
    "solution": {
      "feedback": [
        "Remember to change the sign when moving a term."
      ],
      "lesson": [
        "Rules for solving linear equations"
      ],
      "intervention": [
        "Short exercises focusing on sign changes."
      ]
    }
  },
  {
    "case_id": 4,
    "problem": {
      "topic": "quadratic equations",
      "error_pattern": ["discriminant error", "arithmetic error"],
      "difficulty": "hard",
      "score": 48
    },
    "solution": {
      "feedback": [
        "Review how to calculate  $b^2 - 4ac$ .",
        "Double-check arithmetic."
      ],
      "lesson": [
        "Understanding and calculating the discriminant"
      ],
      "intervention": [
        "Discriminant drills with immediate feedback."
      ]
    }
  },
  {
    "case_id": 5,
    "problem": {
      "topic": "quadratic equations",
      "error_pattern": [
        "wrong formula",
        "sign error",
        "substitution error"
      ]
    }
  }
]

```

```

    ],
    "difficulty": "medium",
    "score": 50
},
"solution": {
    "feedback": [
        "Identify a, b, and c first.",
        "Apply the quadratic formula carefully."
    ],
    "lesson": [
        "Complete quadratic formula tutorial"
    ],
    "intervention": [
        "Worked example + guided practice + error checking."
    ]
}
}
]

# -----
# 2. DISPLAY AVAILABLE ERROR TYPES
# -----

available_errors = [
    "wrong formula",
    "sign error",
    "substitution error",
    "arithmetic error",
    "transposition error",
    "discriminant error"
]

# -----
# 3. GET NEW CASE FROM USER
# -----

def get_new_case(case_id):

    print("\n" + "=" * 60)
    print("ENTER NEW STUDENT CASE")
    print("=" * 60)

    # Topic
    topic = input("Enter math topic: ").strip().lower()

    # Difficulty
    while True:
        difficulty = input(
            "Enter difficulty (easy/medium/hard): "
        ).strip().lower()

        if difficulty in ["easy", "medium", "hard"]:
            break

    print("Invalid difficulty. Please choose easy, medium, or hard.")

    # Score
    while True:
        try:
            score = float(input("Enter student's score (0-100): "))

            if 0 <= score <= 100:
                break

        print("Score must be between 0 and 100.")

```



```

    except ValueError:
        print("Please enter a valid number.")

# Error patterns
print("\nAvailable error patterns:")

for i, error in enumerate(available_errors, start=1):
    print(f"{i}. {error}")

while True:
    error_input = input(
        "\nEnter error numbers separated by commas "
        "(example: 1,2,3): "
    )

    try:
        selected_numbers = [
            int(x.strip())
            for x in error_input.split(",")
        ]

        if all(
            1 <= number <= len(available_errors)
            for number in selected_numbers
        ):
            break

        print("One or more numbers are invalid.")

    except ValueError:
        print("Please enter numbers separated by commas.")

error_pattern = [
    available_errors[number - 1]
    for number in selected_numbers
]

return {
    "case_id": case_id,
    "problem": {
        "topic": topic,
        "error_pattern": error_pattern,
        "difficulty": difficulty,
        "score": score
    }
}

# -----
# 4. SIMILARITY ASSESSMENT
# -----

def calculate_similarity(new_problem, old_problem):

    score = 0

    # Topic = 40%
    if new_problem["topic"] == old_problem["topic"]:
        score += 40

    # Error patterns = 40%
    new_errors = set(new_problem["error_pattern"])
    old_errors = set(old_problem["error_pattern"])

```

```

if new_errors or old_errors:

    intersection = new_errors.intersection(old_errors)
    union = new_errors.union(old_errors)

    error_similarity = len(intersection) / len(union)

    score += error_similarity * 40

# Difficulty = 10%
if new_problem["difficulty"] == old_problem["difficulty"]:
    score += 10

# Score similarity = 10%
score_difference = abs(
    new_problem["score"] - old_problem["score"]
)

score_similarity = max(
    0,
    1 - (score_difference / 100)
)

score += score_similarity * 10

return round(score, 2)

# -----
# 5. RETRIEVE SIMILAR CASES
# -----

def retrieve_cases(new_case):

    results = []

    for case in case_base:

        similarity = calculate_similarity(
            new_case["problem"],
            case["problem"]
        )

        results.append({
            "case": case,
            "similarity": similarity
        })

    # Highest similarity first
    results.sort(
        key=lambda x: x["similarity"],
        reverse=True
    )

    return results

# -----
# 6. ADAPT SOLUTION
# -----

def adapt_solution(new_case, retrieved_case):

    errors = new_case["problem"]["error_pattern"]

```

```

feedback = []
lessons = []
interventions = []

# Customize feedback according to errors
if "wrong formula" in errors:

    feedback.append(
        "Review the correct formula before starting the problem."
    )

    lessons.append(
        "Identifying and selecting the correct formula"
    )

    interventions.append(
        "Practice choosing the correct formula from several examples."
    )

if "sign error" in errors:

    feedback.append(
        "Check positive and negative signs at every step."
    )

    lessons.append(
        "Handling positive and negative signs"
    )

    interventions.append(
        "Complete guided exercises focusing on sign changes."
    )

if "substitution error" in errors:

    feedback.append(
        "Identify a, b, and c before substituting values."
    )

    lessons.append(
        "Correct substitution into formulas"
    )

    interventions.append(
        "Use a step-by-step substitution checklist."
    )

if "arithmetic error" in errors:

    feedback.append(
        "Recheck calculations before submitting the final answer."
    )

    lessons.append(
        "Arithmetic accuracy"
    )

    interventions.append(
        "Complete short calculation drills."
    )

if "transposition error" in errors:

    feedback.append(
        "Remember to change the sign when moving terms."

```

```

    )

    lessons.append(
        "Transposition rules"
    )

    interventions.append(
        "Practice moving terms across the equals sign."
    )

    if "discriminant error" in errors:

        feedback.append(
            "Review the discriminant formula  $b^2 - 4ac$ ."
        )

        lessons.append(
            "Calculating the discriminant"
        )

        interventions.append(
            "Practice calculating discriminants step by step."
        )

    # If an error was not recognized
    if not feedback:
        feedback.append(
            "Review the student's solution step by step."
        )

    # Include successful intervention from retrieved case
    interventions.append(
        "Adapted from the successful intervention used in "
        f"Case {retrieved_case['case_id']}."
    )

    return {
        "feedback": feedback,
        "lesson": lessons,
        "intervention": interventions
    }

# -----
# 7. DISPLAY RESULTS
# -----

def display_results(new_case, ranked_cases, adapted_solution):

    print("\n" + "=" * 60)
    print("CBR ANALYSIS RESULTS")
    print("=" * 60)

    # New problem
    print("\nNEW PROBLEM DESCRIPTION")
    print("-" * 60)

    print("Topic:",
          new_case["problem"]["topic"])

    print("Difficulty:",
          new_case["problem"]["difficulty"])

    print("Score:",
          new_case["problem"]["score"])

```

```
print("Error patterns:",
      ", ".join(new_case["problem"]["error_pattern"]))
```

```
# Ranking
print("\nSIMILARITY RANKING")
print("-" * 60)
```

```
for rank, result in enumerate(ranked_cases, start=1):
```

```
    case = result["case"]
```

```
    print(
        f"{rank}. Case {case['case_id']} "
        f"-> Similarity: {result['similarity']}%"
    )
```

```
# Retrieved case
best_match = ranked_cases[0]["case"]
```

```
print("\nMOST RELEVANT PAST CASE")
print("-" * 60)
```

```
print("Case ID:", best_match["case_id"])
```

```
print(
    "Similarity:",
    ranked_cases[0]["similarity"],
    "%"
)
```

```
print(
    "Topic:",
    best_match["problem"]["topic"]
)
```

```
print(
    "Errors:",
    ", ".join(
        best_match["problem"]["error_pattern"]
    )
)
```

```
# Adapted solution
print("\nADAPTED FINAL SOLUTION")
print("-" * 60)
```

```
print("\nPersonalized Feedback:")
```

```
for item in adapted_solution["feedback"]:
    print(" ✓", item)
```

```
print("\nPersonalized Lessons:")
```

```
for item in adapted_solution["lesson"]:
    print(" ✓", item)
```

```
print("\nRecommended Intervention:")
```

```
for item in adapted_solution["intervention"]:
    print(" ✓", item)
```

```

# -----
# 8. RETAIN NEW CASE
# -----

def retain_case(new_case, adapted_solution):

    # Add the generated solution to the case
    new_case["solution"] = adapted_solution

    # Store the new case
    case_base.append(new_case)

    print("\n" + "=" * 60)
    print("LEARNING / RETAIN STAGE")
    print("=" * 60)

    print("✔ New case saved successfully!")

    print(
        f"✔ Case {new_case['case_id']} added to the case base."
    )

    print(
        f"✔ Updated case base contains "
        f"{len(case_base)} cases."
    )

# -----
# 9. DISPLAY UPDATED CASE BASE
# -----

def display_case_base():

    print("\nUPDATED CASE BASE")
    print("=" * 60)

    for case in case_base:

        print(
            f"Case {case['case_id']} | "
            f"Topic: {case['problem']['topic']} | "
            f"Difficulty: {case['problem']['difficulty']} | "
            f"Score: {case['problem']['score']} "
        )

        print(
            "  Errors:",
            ", ".join(
                case["problem"]["error_pattern"]
            )
        )

# -----
# 10. MAIN PROGRAM
# -----

def main():

    print("=" * 60)
    print("INTELLIGENT TUTORING SYSTEM")
    print("CASE-BASED REASONING")
    print("=" * 60)

```

```

print(
    "\nThe system currently contains",
    len(case_base),
    "past cases."
)

# Dynamically create new case
new_case_id = len(case_base) + 1

new_case = get_new_case(new_case_id)

# Retrieve similar cases
ranked_cases = retrieve_cases(new_case)

# Get best case
best_case = ranked_cases[0]["case"]

# Adapt solution
adapted_solution = adapt_solution(
    new_case,
    best_case
)

# Display results
display_results(
    new_case,
    ranked_cases,
    adapted_solution
)

# Retain new case
retain_case(
    new_case,
    adapted_solution
)

# Display updated case base
display_case_base()

# -----
# RUN PROGRAM
# -----

if __name__ == "__main__":
    main()

```

B.

Title:

A Case-Based Intelligent Tutoring System for Senior High School Algebra, Calculus, and Geometry

Description:

This study proposes a Case-Based Reasoning (CBR) intelligent tutoring system developed in Python to identify and address the learning difficulties of senior high school students in Algebra, Calculus, and Geometry. The system analyzes students' errors, misconceptions, and problem-solving approaches to provide individualized instructional support.

Similarity Method:

The system retrieves previous cases by comparing the current student's problem with stored cases involving similar error patterns, mathematical topics, problem-solving steps, and levels of difficulty. For example, a student who incorrectly applies a formula or misinterprets a geometric concept may be matched with previous students who demonstrated comparable misconceptions. Similarity measures are used to determine which past cases are most relevant to the student's current learning difficulty.

Adaptation Method:

After retrieving the most relevant cases, the system adapts the solutions and interventions based on the current student's specific needs. These interventions may include corrective feedback, guided explanations, worked examples, practice exercises, or personalized lessons. The system does not simply replicate the solution provided to a previous student; instead, it modifies the recommended intervention according to the student's identified errors, performance, and level of understanding. This allows the system to provide more relevant and personalized academic support.

Output:

```
Problems  Output  Debug Console  Terminal  Ports  Comments

=====
ENTER NEW STUDENT CASE
=====
Enter math topic: Quadratic Equations
Enter difficulty (easy/medium/hard): Medium
Enter student's score (0-100): 10

Available error patterns:
1. wrong formula
2. sign error
3. substitution error
4. arithmetic error
5. transposition error
6. discriminant error

Enter error numbers separated by commas (example: 1,2,3): 1, 2, 3, 4, 5, 6

=====
CBR ANALYSIS RESULTS
=====

NEW PROBLEM DESCRIPTION
-----
Topic: quadratic equations
Difficulty: medium
Score: 10.0
Error patterns: wrong formula, sign error, substitution error, arithmetic error, transposition error,
discriminant error

SIMILARITY RANKING
-----
1. Case 5 -> Similarity: 76.0%
2. Case 1 -> Similarity: 68.83%
3. Case 2 -> Similarity: 68.33%
4. Case 4 -> Similarity: 59.53%
5. Case 3 -> Similarity: 17.83%

MOST RELEVANT PAST CASE
-----
Case ID: 5
Similarity: 76.0 %
Topic: quadratic equations
Errors: wrong formula, sign error, substitution error
```


ADAPTED FINAL SOLUTION

Personalized Feedback:

- ✓ Review the correct formula before starting the problem.
- ✓ Check positive and negative signs at every step.
- ✓ Identify a, b, and c before substituting values.
- ✓ Recheck calculations before submitting the final answer.
- ✓ Remember to change the sign when moving terms.
- ✓ Review the discriminant formula $b^2 - 4ac$.

Personalized Lessons:

- ✓ Identifying and selecting the correct formula
- ✓ Handling positive and negative signs
- ✓ Correct substitution into formulas
- ✓ Arithmetic accuracy
- ✓ Transposition rules
- ✓ Calculating the discriminant

Recommended Intervention:

- ✓ Practice choosing the correct formula from several examples.
- ✓ Complete guided exercises focusing on sign changes.
- ✓ Use a step-by-step substitution checklist.
- ✓ Complete short calculation drills.
- ✓ Practice moving terms across the equals sign.
- ✓ Practice calculating discriminants step by step.
- ✓ Adapted from the successful intervention used in Case 5.

LEARNING / RETAIN STAGE

- ✓ New case saved successfully!
- ✓ Case 6 added to the case base.
- ✓ Updated case base contains 6 cases.

UPDATED CASE BASE

Case 1 | Topic: quadratic equations | Difficulty: medium | Score: 55
 Errors: wrong formula, sign error
 Case 2 | Topic: quadratic equations | Difficulty: medium | Score: 60
 Errors: wrong formula, substitution error
 Case 3 | Topic: linear equations | Difficulty: easy | Score: 65
 Errors: transposition error, sign error
 Case 4 | Topic: quadratic equations | Difficulty: hard | Score: 48

Case 5 | Topic: quadratic equations | Difficulty: medium | Score: 50
 Errors: wrong formula, sign error, substitution error
 Case 6 | Topic: quadratic equations | Difficulty: medium | Score: 10.0
 Errors: wrong formula, sign error, substitution error, arithmetic error, transposition error, discriminant error

C.

What each partner contributed

- Both researched first about the case study topic and gathered what setting this program would be appropriate. After deciding that the context of the program will be an intelligent tutoring system for senior high school students that are struggling with calculus, geometry and algebra. Finding scenarios that could be used as reference for the case base that will be used in the program as well as solutions that were used. Using liveshare extension on vs code we are able to code simultaneously and work on divided segments of the program.

What challenges you encountered

- The most challenging part of the project is working with the limited number of cases that we can use. Our program uses 5 case bases as the pool for our Intelligent tutoring system to retrieve from so we need our case bases to contain general and wide problems as well as actual helpful solutions. We aim for diverse case bases so when we input a new problem our recorded case base can have large enough similarity so that their solution can be adapted to the new problem. We settled with at least covering subjects under our selected subject and new problems can reference them. The problems details also contain common student experience or general struggles so any new cases but be able to branch out with more adaptive solutions.

What you learned about CBR

- We learned that Case-Based Reasoning is a problem-solving and machine learning approach wherein a system adapts new solutions for recent problem instances from similar previously solved problems. While it is certainly useful from the perspective of allowing a system to learn from experience rather than developing new strategies from scratch for every new problem, we observed that this also becomes its limitation, as it cannot formulate applicable solutions for problems which are absent from its dataset. As such, we believe that the effectiveness of a CBR system ultimately depends on the size of the dataset.